SHEET 07

KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

Concurrency & Embedded

Fictional ECU-themed examples of C concurrency, POSIX signals and bounded numerical/input policies. Language rules, platform contracts and recorded tool observations have separate scopes.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fanalyzer` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▶ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 10 entries

CON-01 Race condition: check-then-act

SR-404 · Fuel pump (two injector tasks)

✕ BUGGY

```c
static int fuel_ml = 100;
void *pump(void *arg) {
    if (fuel_ml >= 60)          // check ...
        fuel_ml -= 60;  // conflicting access: data race
    return arg;
}
```

▶ TSan: data race

What goes wrong: The two threads access the same non-atomic fuel_ml with conflicting operations and no synchronization: a C11 data race and undefined behavior. Neither a final value of -20 nor lost updates is guaranteed. Replacing separate accesses with atomics can remove that data race while still allowing both tasks to pass the check.

> FIX

```c
static int fuel_ml = 100;
static pthread_mutex_t fuel_lock =
PTHREAD_MUTEX_INITIALIZER;
void *pump(void *arg) {
    check_pthread(pthread_mutex_lock(&fuel_lock));
    if (fuel_ml >= 60)          // check and act under ONE
lock
        fuel_ml -= 60;
    check_pthread(pthread_mutex_unlock(&fuel_lock));
    return arg;
}
```

Avoid it: Author guidance: protect the complete invariant with one mutex, or design a suitable compare-exchange operation with failure handling. The linked driver checks pthread return codes. TSan can diagnose an exercised data race; a clean run is not proof that every interleaving or logic error is safe.

Sources: C11 draft §5.1.2.4 ¶25; §7.17.3; §7.17.7.4; POSIX pthread mutex DESCRIPTION and return values; POSIX2024 RETURN VALUE and error handling; POSIX2024 DESCRIPTION and return values; Clang ThreadSanitizer docs Introduction; usage and detection limits

CON-02 Deadlock from inconsistent lock order

SR-404 · Bus + dashboard locks

✕ BUGGY

```c
void *gauge_task(void *p);      // locks bus, then dash
void *alarm_task(void *p) {
    check_pthread(pthread_mutex_lock(&dash));  // dash,
then bus: opposite order
    check_pthread(pthread_mutex_lock(&bus));
    check_pthread(pthread_mutex_unlock(&bus));
    check_pthread(pthread_mutex_unlock(&dash));
    return p;
}
```

▶ TSan: lock-order-inversion (potential deadlock)

What goes wrong: Taking bus then dash in one task and dash then bus in another permits a circular wait if both run concurrently and each holds its first mutex. This supplied driver actually joins the first task before starting the second: it demonstrates a lock-order detector report, not an observed deadlock or an elapsed-time prediction.

> FIX

```c
void *gauge_task(void *p);      // locks bus, then dash
void *alarm_task(void *p) {     // documented order: bus,
then dash
    check_pthread(pthread_mutex_lock(&bus));
    check_pthread(pthread_mutex_lock(&dash));
    check_pthread(pthread_mutex_unlock(&dash));
    check_pthread(pthread_mutex_unlock(&bus));
    return p;
}
```

Avoid it: Author guidance: define and follow an acyclic lock order, limit critical sections according to requirements, and audit calls that may acquire locks. C++17 scoped_lock uses the standard multiple-lock algorithm, but cannot rule out every unrelated lock cycle in a program. The observed TSan lock-order report is tool-specific and can occur without an actual deadlock.

Sources: POSIX pthread mutex blocking and mutex types; return values; POSIX2024 RETURN VALUE; POSIX2024 DESCRIPTION: wait for termination; C++17 draft [thread.lock.algorithm]; [thread.lock.scoped]; Sanitizer runtime docs detect_deadlocks; observed report is separately bound to the supplied run

CON-03 volatile used as if it were atomic

SR-404 · Wheel-tick counter

✕ BUGGY

```c
static volatile unsigned long ticks;  // volatile is NOT
thread-safe
void *count_ticks(void *p) {
    for (unsigned long i = 0; i < 100000UL; i++)
        ticks++;  // concurrent non-atomic increment: data
race
    return p;
}
```

▶ TSan: data race

What goes wrong: volatile access semantics do not provide the atomicity or inter-thread synchronization needed by ticks++. These conflicting non-atomic accesses form a C11 data race, with undefined behavior rather than a guaranteed lost-count result. What constitutes a volatile access is implementation-defined.

> FIX

```c
static atomic_ulong ticks;        // <stdatomic.h>, C11
void *count_ticks(void *p) {
    for (unsigned long i = 0; i < 100000UL; i++)
        atomic_fetch_add_explicit(&ticks, 1,
memory_order_relaxed);
    return p;
}
```

Avoid it: Author guidance: use atomic operations for the shared counter and an appropriate synchronization design for compound state. Relaxed fetch-add supplies an indivisible counter update here, but does not publish other data or guarantee lock-free implementation. Volatile signal flags have narrow signal rules; hardware-register access needs target-specific contracts, not an "only these uses" language rule.

Sources: C11 draft §5.1.2.4 ¶25; §6.7.3 ¶7; §7.17.3; §7.17.7.5; §7.14.1.1 ¶5; §5.2.4.2.1 minimum unsigned-long range; §7.21.6.1 %lu; GCC volatile documentation implementation-defined accesses; volatile is not a non-volatile-memory barrier; Clang ThreadSanitizer docs data-race detection, not proof of race freedom

CON-04 Unsafe calls inside a POSIX signal handler

SR-404 · Timer-signal exercise, not a hardware ISR

✕ BUGGY

```c
static void on_tick(int sig) {
    printf("tick %d\n", sig);   // not POSIX async-signal-
safe
}
void start_ticks(void) {
    if (signal(SIGALRM, on_tick) == SIG_ERR) {
        perror("signal setup"); exit(EXIT_FAILURE);
    }
    alarm(1);
}
```

▶ gcc -Wanalyzer-unsafe-call-within-signal-handler: call to 'printf' f…

What goes wrong: This example uses a POSIX SIGALRM handler, not a hardware interrupt implementation. printf is not async-signal-safe; calling it from an asynchronously delivered handler can violate the signal-safety contract. Deadlock or corruption is not a guaranteed result, and hardware interrupt latency depends on a separate target contract.

> FIX

```c
static volatile sig_atomic_t tick_pending;
static void on_tick(int sig) {
    (void)sig;
    tick_pending = 1;  // coalesces events; not an event
count
}
/* Install with checked sigaction in the linked fixed
setup.
   Main loop handles the pending flag outside the handler.
*/
```

Avoid it: Author guidance: defer ordinary I/O to normal execution and follow the applicable signal-safe operation rules. POSIX lists write as async-signal-safe, but its descriptor, return value and errno still require care. The volatile sig_atomic_t assignment is a pending-event flag, not a reliable event counter or an arbitrary-thread synchronization mechanism. GCC analyzer diagnosis is a bounded tool observation.

Sources: C11 draft §7.14.1.1 ¶5: asynchronous handler restrictions; POSIX signal rules §2.4.3 Signal Actions: async-signal-safe operation list; POSIX signal signal return value SIG_ERR and POSIX interface scope; POSIX sigaction DESCRIPTION and RETURN VALUE; POSIX sigemptyset DESCRIPTION and RETURN VALUE; POSIX alarm scheduled signal and remaining-time return value; POSIX2024 RETURN VALUE and error conditions; GCC14 docs -Wanalyzer-unsafe-call-within-signal-handler; compiler diagnosis scope

CON-05 Publishing data with relaxed atomics

SR-404 · Sensor frame hand-off

✕ BUGGY

```
void *producer(void *p) {
    frame[0] = 42;
    atomic_store_explicit(&ready, true,
                          memory_order_relaxed);   // no
ordering!
    return p;
}
void *consumer(void *p) {
    while (!atomic_load_explicit(&ready,
memory_order_relaxed)) { }
    printf("frame %d\n", frame[0]);   // stale? data race!
    return p;
}
```

▸ TSan: data race

What goes wrong: The relaxed flag operations are atomic but do not synchronize publication of the ordinary frame element. The producer write and consumer read are not ordered by happens-before, so this program has a C11 data race and undefined behavior. This diagnosis does not depend on observing a stale value on ARM or any other CPU.

> FIX

```
void *producer(void *p) {
    frame[0] = 42;
    atomic_store_explicit(&ready, true,
                          memory_order_release);   //
publish
    return p;
}
void *consumer(void *p) {
    while (!atomic_load_explicit(&ready,
memory_order_acquire)) { }
    printf("frame %d\n", frame[0]);  // after observing
this release: value is 42
    return p;
}
```

Avoid it: Author guidance: for this one-shot example, a release store and an acquire load that reads from it establish the synchronization needed for frame[0]. Reusing a buffer needs an additional ownership/protocol design. Neither this spin loop nor the language promises a response deadline; TSan observations do not prove all schedules.

Sources: C11 draft §5.1.2.4 ¶11, ¶25; §7.17.3: relaxed vs release/acquire; observation of release; POSIX2024 RETURN VALUE; POSIX2024 join contract; Clang ThreadSanitizer docs instrumented race detection and limits

CON-06 Unbounded wait: the watchdog bites

SR-404 · Wheel-speed sensor + ECU watchdog

✕ BUGGY

```
bool sensor_ready(void);  // this demonstration supplies an
always-false stub
int read_wheel_speed(void) {
    while (!sensor_ready()) { }   // no retry bound in this
example
    return sensor_value();
}
```

▸ hangs: killed by the test watchdog

What goes wrong: The supplied sensor stub always returns false, so the bad function does not return in the host demonstration and is killed by the test runner's timeout. Separately, the game ECU watch() faults a busy running worker after three consecutive checks with elapsed time greater than 500 ms; boot uses 5000 ms. Those are game rules, not a precise 500-ms hardware deadline.

> FIX

```
bool sensor_ready(void);
int read_wheel_speed(int *out) {
    if (out == NULL) return -1;
    for (int tries = 0; tries < 1000; tries++) {
        if (sensor_ready()) { *out = sensor_value(); return
0; }
    }
    return -1;  // caller decides how to handle the fault
}
```

Avoid it: Author guidance: define error handling and suitable time bounds for the actual system, including each called operation. This retry loop bounds calls to sensor_ready, not wall-clock duration if that function can block. Returning -1 reports a fault; the caller must implement any fallback. Hardware watchdog service policy needs its actual platform and supervision requirements; use fault injection to test them.

Sources: C11 draft §6.8.5; §6.5.2.2; §6.8.6.4: control flow is not a wall-clock bound; Game ECU runtime WATCHDOG_MS/BOOT_MS; watch >limit and strikes>=3; start timer; _haywire/_kill

CON-07 No input validation or output clamping

SR-404 · Steering actuator

✕ BUGGY

```
float rate_cmd(float wheel, float dt) {
    return wheel / dt * 0.05f;  // shown Annex F samples:
+inf and NaN
}
```

▸ prints: inf nan

What goes wrong: For the stated Annex F build with default non-trapping exceptions, the supplied positive numerator divided by +0 produces infinity and the NaN input produces a NaN result. These are not universal C11-core arithmetic outcomes. The game actuator separately replaces every non-finite scalar output with zero and clamps to the system range; source code alone does not establish real vehicle effects.

> FIX

```
float rate_cmd(float wheel, float dt) {
    if (!isfinite(wheel) || !isfinite(dt) || !(dt > 0.0f))
        return 0.0f;  // exercise policy, not a safety
proof
    float scaled = wheel * 0.05f;
    if (scaled >= dt) return 1.0f;
    if (scaled <= -dt) return -1.0f;
    return scaled / dt;  // magnitude below 1 before
division
}
```

Avoid it: Author guidance: define acceptable inputs and an explicit application output policy. This Annex F example rejects non-finite inputs and non-positive dt, then scales before division and saturates before a potentially overflowing quotient. Tiny inputs can still round or underflow. Build assumptions exclude fast-math and changed floating-point environment modes. A numerical bound or zero default is not evidence of physical safety.

Sources: C11 draft §6.5 ¶5 and §6.5.5 vs Annex F.1/F.3; F.8.3 default exception environment; §7.12.3.2 isfinite; GCC14 docs -ffast-math; -ffinite-math-only; incompatible assumptions excluded here; Game ECU runtime RANGE and _actuator: non-finite scalar outputs become zero and other outputs are bounded

CON-08 Trusting a length field (fuzz your parsers)

SR-404 · CAN / diagnostics-port frames

✕ BUGGY

```
int parse_frame(const uint8_t *buf, size_t n, uint8_t
out[8]) {
    uint8_t len = buf[0];          // comes off the wire
    memcpy(out, buf + 1, len);     // len > 8 or > n - 1:
overflow
    return len;
}
```

▸ ASan: heap-buffer-overflow

libFuzzer harness · instrumented exploration; no discovery-time guarantee

FUZZ

```
int LLVMFuzzerTestOneInput(const uint8_t *d, size_t n) {
    uint8_t out[8];
    if (parse_frame(d, n, out) > 0) {
        volatile uint8_t sink = out[0];  // keep the copy
observable
        (void)sink;
    }
    return 0;
}
```

What goes wrong: Reading buf[0] requires an available input byte. A length exceeding the readable payload or writable output capacity makes the copy invalid; the supplied 200-byte request exceeds both example objects, but no particular failure or diagnosis is guaranteed by C. The array parameter syntax out[8] adjusts to a pointer and does not enforce capacity.

> FIX

```
int parse_frame(const uint8_t *buf, size_t n, uint8_t
out[8]) {
    if (n < 1 || buf == NULL || out == NULL) return -1;
    uint8_t len = buf[0];
    if (len > 8 || len > n - 1) return -1;   // validate
first
    memcpy(out, buf + 1, len);
    return len;
}
```

Avoid it: Author guidance: check lengths before access and document pointer, capacity and non-overlap contracts. The fixed API requires out to provide eight writable bytes and buf to provide the claimed readable bytes, with no overlap for memcpy; integer n alone cannot prove those properties. libFuzzer explores supplied instrumented builds and corpora, with no guaranteed discovery time or exhaustive correctness proof.

Sources: C11 draft §6.7.6.3 ¶7 array parameter adjustment; §6.5.2.1; §7.24.2.1 memcpy contracts; Clang docs instrumented out-of-bounds detection and limitations; Clang libFuzzer docs in-process input exploration and corpus; no time/completeness guarantee

CON-09 Condition variable waited with if, not while

SR-404 · Frame-ready hand-off

✕ BUGGY

```c
void *ecu_consumer(void *p) {
    check_pthread(pthread_mutex_lock(&m));
    if (!frame_ready)
        wait_for_frame();  // checked cond_wait; witness in scaffold
    printf("frame=%d\n", frame);  // predicate may still be false
    check_pthread(pthread_mutex_unlock(&m));
    return p;
}
```

▸ prints: frame=0

What goes wrong: POSIX condition waits may wake spuriously, and a signaled wakeup does not itself establish the application predicate. The supplied driver deliberately signals while frame_ready is false; that is an explicit signal, not an observed spontaneous spurious wakeup. Separate witness coordination replaces timing sleeps.

> FIX

```c
void *ecu_consumer(void *p) {
    check_pthread(pthread_mutex_lock(&m));
    while (!frame_ready)
        wait_for_frame();  // checked cond_wait; witness in scaffold
    printf("frame=%d\n", frame);  // predicate holds with the mutex locked
    check_pthread(pthread_mutex_unlock(&m));
    return p;
}
```

Avoid it: Author guidance: test the predicate under the associated mutex after every successful wakeup and handle errors; C++ predicate wait overloads perform their own loop. This linked driver uses a second condition variable only to coordinate the demonstration. Correct predicate use does not by itself guarantee scheduler progress, producer completion or bounded waiting.

Sources: POSIX condition waits DESCRIPTION: predicate, spurious wakeups, mutex association; RETURN VALUE; POSIX pthread mutex lock/unlock contracts; POSIX2024 return value handling; POSIX2024 join contract; C++17 draft [thread.condition.condvar]: predicate wait overload

CON-10 Tick-counter wraparound in timeouts

SR-404 · ECU watchdog timer

✕ BUGGY

```c
bool timed_out(uint32_t now_ms, uint32_t start_ms) {
    uint32_t deadline = start_ms + 500;  // deadline addition can wrap near UINT32_MAX
    return now_ms >= deadline;  // wrong even before now itself wraps
}
```

▸ prints: timed_out=1 after 50 ms

What goes wrong: An exact 32-bit millisecond counter has period 2^32 ms, about 49.7103 days. The shown start is 100 ticks below UINT32_MAX: adding 500 wraps the deadline, so the bad comparison already reports timeout after only 50 ms, before the counter itself wraps. Unsigned elapsed subtraction represents elapsed time only within a known single-period horizon.

> FIX

```c
bool timed_out(uint32_t now_ms, uint32_t start_ms) {
    return (uint32_t)(now_ms - start_ms) >= 500u;  // one-wrap horizon
}
```

Avoid it: Author guidance: validate that the true elapsed duration is less than 2^32 ticks and poll in time to observe the required timeout. Multiple elapsed periods are indistinguishable from these two timestamps. A wider counter or a supported monotonic clock API may suit longer spans, but clock_gettime does not promise a 64-bit application counter; check return values, clock support, representation and units.

Sources: C11 draft §6.2.5 ¶9; §6.3.1.3; §7.20.1.1: exact-width unsigned type, modulo conversion; 2^32/86400000≈49.7103 is derived; POSIX clock_gettime CLOCK_MONOTONIC clock semantics, timespec, RETURN VALUE and errors

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.